Exercices - Initiation à Python

⋆ application directe ⋆⋆ mise en œuvre ⋆⋆⋆ problème ⋆⋆⋆⋆ problème long, en autonomie

Chapitre 1 - Découvrir Python

Exercice 1 ⋆

Pour chacun des programmes suivants, Python affiche un message d'erreur. Donner le *type* de l'erreur, indiquer la ligne fautive, puis proposer une correction.

```
# programme a
print("Bonjour)
```

```
# programme b
age = 15
print("Vous avez", agee, "ans")
```

```
# programme c
n = input("Un entier : ")
print(n + 1)
```

```
# programme d
if 3 > 2:
print("Trois est plus grand")
```

```
# programme e
a = 10
b = 0
print(a / b)
```

Exercice 2 ⋆

Prédire, *sans machine*, ce que chacune de ces instructions affiche. On distinguera bien les guillemets.

```
print(3 + 4)
print("3 + 4")
print("3" + "4")
print(2 * 5)
print("2" * 5)
print("Bonjour", "Marie")
print("Bonjour" + "Marie")
```

Vérifier ensuite les réponses à la machine, et expliquer les deux résultats qui surprennent le plus.

Exercice 3 ⋆

Le programme suivant calcule le prix TTC d'un article, mais il est illisible.

```
p=45.9
t=0.2
r=p*(1+t)
print(r)
```

1. Exécuter le programme et noter ce qu'il affiche.
2. Le réécrire en donnant aux variables des noms explicites, en ajoutant des commentaires utiles, et en rendant l'affichage compréhensible pour l'utilisateur.
3. Combien de commentaires avez-vous écrits ? Un commentaire tel que `# on multiplie p par 1+t` vous paraît-il utile ? Justifier.

Exercice 4 ⋆⋆

On travaille d'abord dans la *console*, puis dans un *script*.

1. Dans la console, on saisit successivement les trois lignes suivantes. Qu'affiche la console à chaque fois ?

```
2 + 3
x = 7
x * 2
```

2. On écrit maintenant ces trois mêmes lignes dans un script, que l'on exécute. Qu'affiche l'écran ? Expliquer la différence.
3. Modifier le script pour qu'il affiche les mêmes résultats que la console.
4. Dans quel cas la console est-elle préférable au script ? Et l'inverse ?

Chapitre 2 - Variables, types et calculs

Exercice 5 ⋆

Recopier et compléter le tableau de trace du programme ci-dessous, en indiquant la valeur de chaque variable après chaque ligne.

```
a = 5
b = 3
a = a + b
b = a - b
a = a - b
print(a, b)
```

Ligne	a	b
1	5	—
2		
3		
4		
5		

Que réalise ce programme ? Écrire une version plus courte qui produit le même effet.

Exercice 6 ⋆

1. Parmi les noms suivants, lesquels sont des noms de variables valides en Python ? Pour les autres, préciser pourquoi.

`prix` `2eme_note` `note2` `prix total` `prix_total` `print` `Prix` `if` `_x`

2. Parmi les noms valides, lesquels sont de *bons* noms pour stocker la moyenne d'un élève ? Justifier.
3. Que se passe-t-il si l'on écrit `print = 5` puis `print("Bonjour")` ?

Exercice 7 ⋆

Donner, sans machine, le type de chacune des expressions suivantes, puis sa valeur.

```
7 / 2
7 // 2
7 % 2
7.0 // 2
```

```
2 ** 10
int("12") + 3
float("3.5") * 2
str(12) + "3"
int(7 / 2)
round(7 / 2)
```

Deux de ces expressions donnent la même valeur mais pas le même type : lesquelles ?

Exercice 8 ⋆⋆

1. Écrire un programme qui demande à l'utilisateur un nombre de minutes, puis affiche la conversion en heures et minutes. Par exemple, pour 200 minutes, il affichera `3 h 20 min`.
2. Adapter le programme pour qu'il traite un nombre de secondes et affiche des heures, des minutes et des secondes.
3. Tester avec 3 725 secondes. Le résultat attendu est `1 h 2 min 5 s`.

Exercice 9 ⋆⋆

1. Écrire un programme qui demande les coordonnées de deux points A et B du plan, puis affiche la distance AB. On importera le module `math`.
2. Améliorer l'affichage pour n'afficher que deux décimales.
3. Ajouter le calcul et l'affichage des coordonnées du milieu de $[AB]$.
4. Tester avec $A(1\,;2)$ et $B(4\,;6)$: la distance doit valoir 5.

Exercice 10 ⋆⋆⋆

Un commerçant applique deux remises successives sur un article : d'abord 20 %, puis 10 % sur le prix déjà réduit.

1. Écrire un programme qui demande le prix initial et affiche le prix final.
2. Un client affirme que cela revient à une remise de 30 %. A-t-il raison ? Modifier le programme pour qu'il affiche également le pourcentage de remise *global*.
3. Ce pourcentage global dépend-il du prix initial ? Tester avec plusieurs valeurs, puis le justifier par le calcul.
4. Modifier le programme pour qu'il demande les deux taux de remise à l'utilisateur.

Chapitre 3 - Tests et conditions

Exercice 11 ⋆

Donner la valeur — `True` ou `False` — de chacune des expressions suivantes, sachant que `a = 5`, `b = 12` et `mot = "python"`.

```
a < b
a == 5
a != b
a > 3 and b > 15
a > 3 or b > 15
not (a == b)
a < b < 20
mot == "Python"
len(mot) == 6
a % 2 == 0
```

Deux de ces lignes surprennent souvent : `mot == "Python"` et `a < b < 20`. Expliquer la valeur de chacune.

Exercice 12 ⋆

1. Écrire un programme qui demande un nombre à l'utilisateur et affiche s'il est positif, négatif ou nul.
2. Écrire un programme qui demande un entier et affiche s'il est pair ou impair.
3. Écrire un programme qui demande trois nombres et affiche le plus grand. On n'utilisera *pas* la fonction `max`.

Exercice 13 ⋆⋆

Le programme suivant devrait afficher la mention obtenue au baccalauréat, mais il comporte *quatre* erreurs.

```
note = input("Votre note : ")
if note >= 16
    print("Mention très bien")
elif note >= 14:
print("Mention bien")
elif note >= 12:
    print("Mention assez bien")
else:
    print("Pas de mention")
```

1. Repérer les quatre erreurs et préciser, pour chacune, le message que Python afficherait.
2. Écrire la version corrigée.
3. Tester avec les notes 16, 14, 12 et 8. Que se passe-t-il avec la note 20 ? Et avec -3 ?
4. Ajouter un test qui refuse les notes hors de l'intervalle $[0\,;20]$.

Exercice 14 ⋆⋆

1. Écrire un programme qui demande trois longueurs et indique si elles peuvent être celles des côtés d'un triangle. On rappelle que c'est le cas lorsque chaque longueur est inférieure à la somme des deux autres.
2. Compléter le programme pour qu'il précise, le cas échéant, si le triangle est rectangle, en utilisant la réciproque du théorème de Pythagore.
3. Le compléter encore pour qu'il indique si le triangle est isocèle ou équilatéral.
4. Tester avec $(3\,;4\,;5)$, $(2\,;2\,;2)$, $(1\,;1\,;5)$ et $(5\,;5\,;7)$.

Exercice 15 ⋆⋆⋆

Une année est bissextile lorsqu'elle est divisible par 4, *sauf* si elle est divisible par 100 sans l'être par 400.

1. Les années 1900, 2000, 2023 et 2024 sont-elles bissextiles ? Répondre à la main.
2. Écrire un programme qui demande une année et affiche si elle est bissextile, en utilisant une suite de `if` et `elif`.
3. Écrire une seconde version qui n'utilise qu'*un seul* test, à l'aide des opérateurs `and`, `or` et `not`.
4. Vérifier que les deux versions donnent les mêmes réponses sur les quatre années de la question 1, puis sur toutes les années de 1890 à 1910.

Chapitre 4 - Les boucles

Exercice 16 ⋆

Prédire ce qu'affiche chacune des boucles suivantes, puis vérifier à la machine.

```
for i in range(5):
    print(i)
```

```
for i in range(1, 5):
    print(i)
```

```
for i in range(0, 10, 3):
    print(i)
```

```
for i in range(5, 0, -1):
    print(i)
```

```
for i in range(3):
    print("*", end="")
```

Combien de valeurs `range(a, b)` produit-elle, en fonction de `a` et `b` ?

Exercice 17 ⋆

1. Écrire un programme qui affiche la table de multiplication de 7, de 7×1 à 7×10.
2. Le modifier pour qu'il demande à l'utilisateur la table qu'il souhaite afficher.
3. Écrire un programme qui calcule et affiche la somme des entiers de 1 à 100.
4. Écrire un programme qui calcule le produit des entiers de 1 à 10, c'est-à-dire 10!.

Exercice 18 ⋆⋆

On considère la suite définie par $u_0 = 3$ et, pour tout entier n, $u_{n+1} = 2u_n - 5$.

1. Calculer à la main u_1, u_2 et u_3.
2. Écrire un programme qui affiche les dix premiers termes de la suite.
3. Modifier le programme pour qu'il demande n à l'utilisateur et affiche uniquement u_n.
4. La suite semble-t-elle croissante ? Que se passe-t-il si l'on remplace $u_0 = 3$ par $u_0 = 5$? Par $u_0 = 6$? Commenter.

Exercice 19 ⋆⋆

1. Écrire un programme qui affiche le motif suivant, à l'aide de deux boucles imbriquées :

```
*
**
***
****
*****
```

2. Modifier le programme pour que le nombre de lignes soit demandé à l'utilisateur.
3. Écrire un programme qui affiche le motif inverse, qui commence par cinq étoiles et termine par une.
4. Afficher enfin un triangle aligné à droite, chaque ligne étant complétée par des espaces à gauche.

Exercice 20 ⋆⋆

1. Écrire un programme qui, à l'aide d'une boucle `while`, détermine le plus petit entier n tel que $2^n > 1\,000\,000$.
2. On place 1 000 euros sur un compte rémunéré à 3 % par an. Écrire un programme qui détermine au bout de combien d'années le capital dépasse 2 000 euros.
3. Le programme précédent afficherait-il la même réponse avec une boucle `for` ? Expliquer pourquoi la boucle `while` s'impose ici.
4. Modifier le programme pour qu'il affiche également le capital obtenu, arrondi au centime.

Exercice 21 ⋆⋆

Le programme suivant ne s'arrête jamais.

```
n = 10
while n > 0:
    print(n)
    n = n + 1
```

1. Expliquer pourquoi.
2. Le corriger pour qu'il affiche les entiers de 10 à 1.
3. Voici un autre programme. Combien de fois la boucle s'exécute-t-elle ?

```
x = 100
compteur = 0
while x > 1:
    x = x / 2
    compteur = compteur + 1
print(compteur)
```

4. Que se passerait-il si l'on remplaçait `x > 1` par `x > 0` ? Justifier.

Exercice 22 ⋆⋆⋆

1. Écrire un programme qui demande un entier $n \geqslant 2$ et affiche tous ses diviseurs.
2. En déduire un programme qui indique si n est premier.
3. Améliorer le programme précédent pour qu'il s'arrête dès qu'un diviseur est trouvé, et ne teste que les entiers jusqu'à $\sqrt{n}$. Justifier que cela suffit.
4. Écrire un programme qui affiche tous les nombres premiers inférieurs à 100.
5. Combien de tests le programme de la question 2 effectue-t-il pour $n = 9\,973$? Et celui de la question 3 ?

Chapitre 5 - Les fonctions

Exercice 23 ⋆

1. Écrire une fonction `aire_rectangle(L, l)` qui renvoie l'aire d'un rectangle.
2. Écrire une fonction `perimetre_cercle(r)` qui renvoie le périmètre d'un cercle de rayon `r`.
3. Écrire une fonction `celsius_vers_fahrenheit(c)`, sachant que $f = \dfrac{9c}{5} + 32$.
4. Tester les trois fonctions, puis les utiliser dans un petit programme qui dialogue avec l'utilisateur.

Exercice 24 ⋆

On considère les deux fonctions suivantes.

```
def double_a(x):
    return 2 * x

def double_b(x):
    print(2 * x)
```

1. Qu'affiche `print(double_a(5))` ? Et `print(double_b(5))` ?
2. Que vaut `double_a(5) + 1` ? Et `double_b(5) + 1` ?
3. Laquelle des deux fonctions est réutilisable dans un calcul ? Pourquoi ?
4. Que renvoie une fonction qui ne contient aucun `return` ?

Exercice 25 ⋆⋆

Prédire l'affichage de chacun de ces programmes, *sans machine*, puis vérifier.

```
# programme a
x = 10
def f():
    x = 3
    print(x)
f()
print(x)
```

```
# programme b
def g(n):
    n = n + 1
    return n
a = 5
g(a)
print(a)
```

```
# programme c
def h():
    print(y)
y = 7
h()
```

Expliquer chaque résultat en une phrase, en utilisant le mot *portée*.

Exercice 26 ⋆⋆

1. Écrire une fonction `est_pair(n)` qui renvoie `True` ou `False`.
2. Écrire une fonction `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif. On pourra utiliser les opérateurs `%` et `//`.
3. Écrire une fonction `est_divisible_par_3(n)` qui utilise la fonction précédente, plutôt que l'opérateur `%`.
4. Vérifier sur les entiers de 1 à 50 que cette fonction donne le même résultat que le test `n % 3 == 0`.

Exercice 27 ⋆⋆⋆

1. Écrire une fonction `pgcd(a, b)` renvoyant le PGCD de deux entiers positifs, en appliquant l'algorithme d'Euclide.
2. Écrire une fonction `ppcm(a, b)`, en utilisant la relation $\text{pgcd}(a\,;b) \times \text{ppcm}(a\,;b) = a \times b$.
3. Écrire une fonction `simplifie(a, b)` qui renvoie les deux termes de la fraction $\frac{a}{b}$ rendue irréductible.
4. Tester avec $\frac{252}{300}$, puis avec $\frac{1001}{1430}$.
5. Écrire une fonction `sont_premiers_entre_eux(a, b)` en réutilisant `pgcd`.

Chapitre 6 - Les listes

Exercice 28 ⋆

On pose `notes = [12, 8, 15, 6, 17, 9]`.

1. Que valent `notes[0]`, `notes[3]` et `notes[-1]` ?
2. Que vaut `len(notes)` ? Quel est l'indice du dernier élément ?
3. Que se passe-t-il si l'on écrit `notes[6]` ?
4. Que valent `notes[1:4]`, `notes[:3]` et `notes[3:]` ?
5. Après `notes[1] = 14`, que contient la liste ? Et après `notes.append(11)` ?

Exercice 29 ⋆

1. Écrire un programme qui construit, à l'aide d'une boucle, la liste des carrés des entiers de 1 à 20.
2. Écrire un programme qui construit la liste des 15 premiers termes de la suite définie par $v_0 = 1$ et $v_{n+1} = 3v_n + 2$.
3. Écrire un programme qui, à partir d'une liste de nombres, construit la liste de leurs opposés.
4. Écrire un programme qui construit la liste des entiers de 1 à 50 qui sont divisibles par 3 ou par 5.

Exercice 30 ⋆⋆

Les deux boucles suivantes parcourent la même liste.

```
for note in notes:
    print(note)
```

```
for i in range(len(notes)):
    print(notes[i])
```

1. Affichent-elles la même chose ?
2. On souhaite maintenant afficher `"La note numéro 3 vaut 15"` pour chaque note. Laquelle des deux boucles faut-il utiliser ? Écrire le programme.
3. On souhaite ajouter 1 point à chaque note. Écrire le programme. Laquelle des deux boucles convient ?
4. Énoncer une règle simple permettant de choisir entre les deux formes.

Exercice 31 ⋆⋆

On exécute le programme suivant.

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
print(b)
```

1. Prédire l'affichage, puis vérifier à la machine. Expliquer.
2. Modifier le programme pour que `a` reste inchangée.
3. Même question avec le programme suivant. Y a-t-il un piège ?

```
x = 5
y = x
y = y + 1
print(x)
```

4. Pourquoi le comportement diffère-t-il entre les deux programmes ?

Exercice 32 ⋆⋆

Écrire, sans utiliser les fonctions `sum`, `max`, `min` ni `len`, les fonctions suivantes, chacune prenant une liste de nombres en paramètre.

1. `somme(L)`, qui renvoie la somme des éléments.
2. `longueur(L)`, qui renvoie le nombre d'éléments.
3. `moyenne(L)`, qui renvoie la moyenne, en réutilisant les deux précédentes.
4. `minimum(L)`, qui renvoie le plus petit élément.
5. `compte(L, x)`, qui renvoie le nombre d'occurrences de `x` dans `L`.
6. `inverse(L)`, qui renvoie une *nouvelle* liste contenant les éléments de `L` dans l'ordre inverse.

Exercice 33 ⋆⋆⋆

Un professeur dispose des notes d'une classe dans une liste.

1. Écrire une fonction `effectifs(notes)` qui renvoie une liste de 21 nombres, celui d'indice k donnant le nombre d'élèves ayant obtenu la note k.
2. Écrire une fonction `mediane(notes)` qui renvoie la médiane. On pourra commencer par trier la liste.
3. Écrire une fonction `au_dessus(notes)` qui renvoie le nombre d'élèves dont la note est strictement supérieure à la moyenne de la classe.
4. Tester l'ensemble sur la liste `[12, 8, 15, 6, 17, 9, 11, 14, 5, 13]`.
5. Le nombre d'élèves au-dessus de la moyenne peut-il dépasser la moitié de l'effectif ? Donner un exemple ou démontrer que c'est impossible.

Chapitre 7 - Quelques algorithmes classiques

Exercice 34 ⋆⋆

1. Écrire une fonction `maximum(L)` qui renvoie le plus grand élément d'une liste non vide.
2. Modifier la fonction pour qu'elle renvoie non pas le maximum, mais son *indice*.
3. Que doit renvoyer la fonction si le maximum apparaît plusieurs fois ? Préciser le choix fait par votre programme.
4. Écrire une fonction `deux_plus_grands(L)` qui renvoie les deux plus grandes valeurs d'une liste, sans trier la liste et en ne la parcourant qu'une seule fois.

Exercice 35 ⋆⋆

On cherche une valeur approchée de la solution de $x^3 + x - 1 = 0$ sur l'intervalle $[0\,;1]$.

1. Vérifier que la fonction $f : x \mapsto x^3 + x - 1$ est croissante sur $[0\,;1]$, et que $f(0) < 0 < f(1)$.
2. Écrire un programme qui balaie $[0\,;1]$ avec un pas de 0,01 et affiche un encadrement de la solution.
3. Modifier le programme pour que le pas soit demandé à l'utilisateur.
4. Combien d'itérations le balayage effectue-t-il pour obtenir un encadrement d'amplitude 10^{-6} ?

Exercice 36 ⋆⋆⋆

On reprend l'équation $x^3 + x - 1 = 0$ sur $[0\,;1]$, cette fois par dichotomie.

1. Écrire une fonction `dichotomie(a, b, precision)` qui renvoie un encadrement de la solution d'amplitude inférieure à `precision`.
2. Combien d'étapes sont nécessaires pour une précision de 10^{-6} ? Comparer avec le nombre d'itérations du balayage de l'exercice 35.
3. Modifier la fonction pour qu'elle affiche également le nombre d'étapes effectuées.
4. L'amplitude est divisée par 2 à chaque étape. En déduire, *par le calcul*, le nombre d'étapes nécessaires pour passer de 1 à 10^{-6}, puis vérifier avec le programme.

Exercice 37 ⋆⋆⋆

1. Écrire, selon l'algorithme du tri par sélection, une fonction `tri_selection(L)` qui trie une liste par ordre croissant.
2. Faire afficher la liste après chaque échange, et vérifier le déroulement sur `[5, 2, 8, 1, 9, 3]`.
3. Combien de comparaisons le tri effectue-t-il sur une liste de n éléments ? Vérifier sur des listes de tailles 5, 10 et 20 en ajoutant un compteur.
4. Modifier la fonction pour qu'elle trie dans l'ordre décroissant.

Exercice 38 ⋆⋆

1. Écrire un programme qui simule 1 000 lancers d'un dé à six faces et affiche la fréquence d'apparition de chaque face. On utilisera `from random import randint`.
2. Les six fréquences sont-elles égales ? Relancer plusieurs fois le programme et commenter.
3. Reprendre avec 100 000 lancers. Que constate-t-on ?

4. Écrire un programme qui simule le lancer de *deux* dés et estime la probabilité d'obtenir une somme égale à 7. Comparer à la valeur théorique $\frac{1}{6}$.

Problèmes

Exercice 39 ⋆⋆⋆

Le nombre mystère. L'ordinateur choisit au hasard un entier compris entre 1 et 100, et l'utilisateur doit le deviner.

1. Écrire un programme qui choisit le nombre, puis demande des propositions à l'utilisateur en répondant « trop grand », « trop petit » ou « gagné », jusqu'à ce qu'il trouve.
2. Ajouter un compteur de tentatives, affiché à la fin.
3. Limiter le nombre de tentatives à sept. Afficher le nombre cherché si l'utilisateur échoue.
4. *Inverser les rôles.* Écrire un programme dans lequel c'est l'*ordinateur* qui devine un nombre choisi par l'utilisateur, celui-ci répondant par `"+"`, `"-"` ou `"="`. Quelle stratégie l'ordinateur doit-il suivre pour gagner à coup sûr en au plus sept coups ?
5. Justifier le nombre sept, en reprenant le raisonnement de la dichotomie.

Exercice 40 ⋆⋆⋆

Le chiffre de César. Pour chiffrer un message, on décale chaque lettre de k rangs dans l'alphabet, en revenant au début après `z`.

1. Les fonctions `ord` et `chr` convertissent une lettre en son code et réciproquement. Que valent `ord("a")` et `chr(100)` ?
2. Écrire une fonction `decale(lettre, k)` qui renvoie la lettre décalée de `k` rangs. On ne traitera que les minuscules, et l'on prendra garde au retour au début de l'alphabet.
3. Écrire une fonction `chiffre(message, k)` qui chiffre un message entier. Les caractères qui ne sont pas des minuscules seront laissés inchangés.
4. Écrire la fonction `dechiffre(message, k)`. Comment s'écrit-elle en une ligne à partir de la précédente ?
5. Le message `"dpvu tvs mf uspof"` a été chiffré avec $k = 1$. Le déchiffrer.
6. *Casser le code.* Écrire un programme qui, ne connaissant pas k, affiche les 25 déchiffrements possibles d'un message. Tester sur `"yhql ylgl ylfl"`.

Exercice 41 ⋆⋆⋆⋆

La suite de Syracuse. On part d'un entier $u_0 \geqslant 1$, puis : si u_n est pair, on pose $u_{n+1} = \frac{u_n}{2}$; sinon $u_{n+1} = 3u_n + 1$. On admet que la suite finit toujours par atteindre 1.

1. Calculer à la main les termes de la suite pour $u_0 = 6$, jusqu'à atteindre 1.
2. Écrire une fonction `syracuse(n)` qui renvoie la liste des termes, de $u_0 = n$ jusqu'au premier 1.
3. Le *temps de vol* est le nombre d'étapes nécessaires pour atteindre 1, et l'*altitude maximale* est le plus grand terme rencontré. Écrire deux fonctions les calculant.
4. Déterminer, pour n variant de 1 à 1 000, la valeur de n dont le temps de vol est le plus grand, ainsi que ce temps de vol.
5. Même question pour l'altitude maximale. La valeur de n trouvée est-elle la même ?

6. Tracer, ou décrire, l'allure du nuage de points donnant le temps de vol en fonction de n. Que remarque-t-on de surprenant ?

Exercice 42 ⋆⋆⋆⋆

Estimer π par la méthode de Monte-Carlo. On tire au hasard des points dans le carré $[0\,;1] \times [0\,;1]$, et l'on compte ceux qui tombent dans le quart de disque de centre l'origine et de rayon 1.

1. Quelle est l'aire du carré ? Celle du quart de disque ? En déduire la proportion attendue de points tombant dans le quart de disque.
2. Écrire un programme qui tire n points au hasard à l'aide de `random()`, compte ceux qui vérifient $x^2 + y^2 \leqslant 1$, et en déduit une estimation de π.
3. Tester avec $n = 100$, $n = 10\,000$ puis $n = 1\,000\,000$. Combien de décimales exactes obtient-on à chaque fois ?
4. Relancer trois fois le programme avec $n = 10\,000$. Les estimations sont-elles identiques ? Commenter.
5. On souhaite obtenir π à 10^{-3} près. Cette méthode vous paraît-elle efficace ? Comparer avec la méthode des rectangles du cours.

Exercice 43 ⋆⋆⋆⋆

Le crible d'Ératosthène. Pour trouver tous les nombres premiers inférieurs à N, on écrit la liste des entiers de 2 à N, puis on barre les multiples de 2, puis ceux de 3, et ainsi de suite.

1. Appliquer le crible à la main pour $N = 30$, et donner la liste obtenue.
2. Écrire une fonction `crible(N)` qui renvoie la liste des nombres premiers inférieurs ou égaux à N. On pourra utiliser une liste de booléens de longueur $N + 1$.
3. Vérifier que `crible(100)` renvoie 25 nombres. Combien y en a-t-il jusqu'à 1 000 ?
4. Comparer le temps d'exécution du crible et celui du test de primalité de l'exercice 22, appliqué à chaque entier, pour $N = 10\,000$.
5. *Conjecture de Goldbach.* Tout entier pair supérieur à 2 semble être la somme de deux nombres premiers. Écrire une fonction `goldbach(n)` qui renvoie un tel couple.
6. Vérifier la conjecture pour tous les entiers pairs de 4 à 1 000. En convenant de retenir le couple dont le *premier* terme est le plus petit possible, pour quel entier n ce premier terme est-il le plus grand ?